

Sections 6–9: Installation, SIEM Setup, Windows & Linux Logging

6. Installation Guide

6.1 Creating VMs (VirtualBox, repeat per VM)

1. **Create the host-only network first** (do this before creating any VM): **File** → **Host Network Manager** → **Create** → note the adapter name (e.g. **vboxnet0**), set it to **192.168.56.1/24**, **disable its DHCP server** (you're assigning static IPs manually — DHCP here causes IP drift that breaks Wazuh agent registration later).
2. **New VM wizard**, per host: | VM | Type | Version | RAM | vCPU | Disk | |---|---|---|---|---|---|
| Wazuh-SIEM | Linux | Ubuntu 64-bit | 8192 MB | 4 | 50 GB (dynamically allocated) | |
| MISP | Linux | Ubuntu 64-bit | 4096 MB | 2 | 20 GB | | Win10-Victim | Windows | Windows
10/11 64-bit | 4096 MB | 2 | 60 GB | | Linux-Server | Linux | Ubuntu 64-bit | 2048 MB | 1 |
20 GB | | Kali-Attacker | Linux | Debian 64-bit | 4096 MB | 2 | 40 GB |
3. **Network adapters, per VM** (**Settings** → **Network**):
 - Adapter 1: **Host-only Adapter**, name = your **vboxnet0** — this is the primary/always-on interface.
 - Adapter 2 (Wazuh, MISP, Linux-Server, Kali only — for initial package installs): **NAT** — disable after setup is complete for the attacker/victim VMs.
4. **Attach installer ISO**, boot, install OS normally (standard install, nothing lab-specific at this stage).
5. **Snapshot immediately after clean OS install**, before any tool installation:

VirtualBox Manager → select VM → Snapshots tab → Take Snapshot → name: "00-clean-install"

Take a second snapshot after each major phase ("**01-siem-installed**", "**02-agents-connected**", etc.). This is not optional — you will break something running Mimikatz-class tools or misconfiguring **ossec.conf**, and rebuilding from scratch costs hours you don't have.

6.2 Static IP Configuration

Ubuntu Server (`/etc/netplan/00-installer-config.yaml` — adjust interface name via `ip a`):

network:

version: 2

ethernets:

enp0s3:

dhcp4: no

addresses: [192.168.56.5/24]

routes:

- to: default

via: 192.168.56.1

nameservers:

addresses: [8.8.8.8, 1.1.1.1]

Apply: `sudo netplan apply`

Windows 10/11: Control Panel → Network and Sharing Center → Change adapter settings → right-click host-only NIC → Properties → IPv4 → Use the following IP address

IP address: 192.168.56.10

Subnet mask: 255.255.255.0

Default gateway: 192.168.56.1

Preferred DNS: 8.8.8.8

Kali Linux: `sudo nano /etc/network/interfaces` or use NetworkManager GUI, same scheme, `192.168.56.20`.

6.3 Hostname Configuration

Ubuntu

```
sudo hostnamectl set-hostname wazuh-siem # repeat per VM: misp-server, linux-endpoint
```

```
sudo nano /etc/hosts # add: 127.0.1.1 wazuh-siem
```

Windows: `Settings → System → About → Rename this PC` → reboot to apply.

6.4 Time Synchronization (critical — do not skip)

SIEM correlation depends entirely on accurate, synchronized timestamps across every host. A 5-minute clock drift between Windows and the Wazuh server will make your "mean time to alert" metric meaningless and can break TLS handshakes.

Ubuntu:

```
sudo apt install chrony -y
```

```
sudo systemctl enable --now chrony
```

```
timedatectl set-timezone UTC # standardize all VMs to UTC, not local time
```

```
chronyc tracking # verify sync
```

Windows:

```
w32tm /config /manualpeerlist:"pool.ntp.org" /syncfromflags:manual /reliable:yes /update
```

```
w32tm /resync
```

Set all VMs to **UTC**, not your local timezone — this avoids ambiguity later when correlating a Kali attack timestamp against a Wazuh alert timestamp across VMs that might otherwise have different local-time settings.

7. SIEM Setup — Wazuh + Elastic (Wazuh's bundled OpenSearch/Kibana-equivalent)

7.1 Install (all-in-one: indexer + manager + dashboard)

```
curl -sO https://packages.wazuh.com/4.14/wazuh-install.sh
```

```
sudo bash ./wazuh-install.sh -a
```

What each flag/line does:

- `curl -sO` — silently downloads the script, saving with its original filename.
- `-a` — "all-in-one" install: deploys the Wazuh indexer (OpenSearch fork, stores/indexes all events), the Wazuh manager (rule engine, agent registration), and the Wazuh dashboard (Kibana-fork web UI) on a single host.
- On completion, the script prints the dashboard URL and an auto-generated `admin` password — **copy this now**, it is shown exactly once.

Recover the password later if missed:

```
sudo tar -O -xvf wazuh-install-files.tar wazuh-install-files/wazuh-passwords.txt
```

Access: `https://192.168.56.5` (self-signed cert — browser warning is expected, not an error).

7.2 Freeze the repo (prevents an `apt upgrade` from breaking mid-project)

Debian/Ubuntu:

```
sudo sed -i "s/^deb /#deb /" /etc/apt/sources.list.d/wazuh.list
```

7.3 Indexing — how logs become searchable

Wazuh manager writes every parsed alert into the Wazuh indexer as a JSON document, indexed daily (`wazuh-alerts-4.x-YYYY.MM.DD`). You do not manually create these indices — the manager's `filebeat` output does this automatically once the agent pipeline is live.

Verify:

```
curl -k -u admin:<PASSWORD> https://localhost:9200/_cat/indices?v
```

You should see `wazuh-alerts-*` indices growing in `docs.count` as agents report in.

7.4 Deploy the Windows agent

From the dashboard: **Agents** → **Deploy new agent** → **Windows** → it generates a PowerShell command bound to your server's IP and a one-time enrollment key. Run it **as Administrator** on the Windows VM (command is unique per install — do not reuse one from documentation, it must come from your own dashboard):

```
Invoke-WebRequest -Uri https://packages.wazuh.com/4.x/windows/wazuh-agent-4.14.0-1.msi  
-OutFile $env:tmp\wazuh-agent.msi
```

```
msiexec.exe /i $env:tmp\wazuh-agent.msi /q WAZUH_MANAGER='192.168.56.5'  
WAZUH_REGISTRATION_SERVER='192.168.56.5'
```

```
NET START WazuhSvc
```

Confirm: dashboard → **Agents** list → status = **Active**.

7.5 Deploy the Linux agent

```
curl -o wazuh-agent.deb  
https://packages.wazuh.com/4.x/apt/pool/main/w/wazuh-agent/wazuh-agent_4.14.0-1_amd64.d  
eb \
```

```
&& sudo WAZUH_MANAGER='192.168.56.5' dpkg -i ./wazuh-agent.deb
```

```
sudo systemctl daemon-reload
```

```
sudo systemctl enable --now wazuh-agent
```

7.6 Dashboards (built-in, no manual creation needed initially)

Wazuh dashboard ships pre-built views: **Security Events**, **Integrity Monitoring**, **MITRE ATT&CK**, **Vulnerabilities**. Custom dashboards (Section 14) are built later in Kibana's visualization editor once you have real alert data to work with — building dashboards against empty indices is a wasted step.

7.7 Alerting

Wazuh alerts by default at rule-level severity (**level 0–15**). To route alerts (e.g., email on **level >= 12**), edit `/var/ossec/etc/ossec.conf` on the manager:

```
<global>
```

```
<email_notification>yes</email_notification>
```

```
<smtp_server>smtp.gmail.com</smtp_server>
```

```
<email_from>soc-lab@yourdomain.local</email_from>
```

```
<email_to>you@example.com</email_to>
```

```
</global>
```

For a lab, this is optional — dashboard-based triage is sufficient and safer than routing lab-generated alerts to a real inbox by mistake.

8. Windows Logging (Sysmon)

8.1 Install

Download Sysinternals Sysmon + SwiftOnSecurity baseline config

```
Invoke-WebRequest -Uri https://download.sysinternals.com/files/Sysmon.zip -OutFile  
Sysmon.zip
```

```
Expand-Archive Sysmon.zip -DestinationPath Sysmon
```

```
Invoke-WebRequest -Uri  
https://raw.githubusercontent.com/SwiftOnSecurity/sysmon-config/master/sysmonconfig-export.  
xml -OutFile sysmonconfig-export.xml
```

```
.\Sysmon\Sysmon64.exe -accepteula -i sysmonconfig-export.xml
```

-accepteula skips the interactive license prompt (required for scripted install); **-i** installs with the given config, which defines which event types to log and which to filter out (SwiftOnSecurity's config is tuned to reduce noise from normal Windows/Office activity while keeping security-relevant events).

8.2 Key Event IDs (this is the core reference you'll use constantly)

Event ID	Meaning	Why it matters for detection
1	Process creation	The single most important event — captures full command line, parent process, hashes
2	File creation time changed	Anti-forensic technique (timestomping) indicator
3	Network connection	Process-level network connection — ties an outbound connection to the exact process that made it
5	Process terminated	Useful for process lifetime correlation
6	Driver loaded	Kernel driver loads — relevant for rootkit/EDR-evasion detection
7	Image loaded (DLL)	DLL injection / sideloading detection
8	CreateRemoteThread	Classic process injection indicator (used by Mimikatz-class tools)
10	ProcessAccess	Captures LSASS access attempts — the event ID for credential dumping detection
11	FileCreate	New file written to disk — dropper/payload detection
12/13/14	Registry event (create/modify/delete)	Persistence via Run keys, services
15	FileCreateStreamHash	Alternate Data Stream creation — download-marker evasion

Event ID	Meaning	Why it matters for detection
17/18	Pipe created/connected	Named pipe abuse — common in C2 frameworks (Cobalt Strike-style)
22	DNSEvent	DNS query logging — DNS tunneling / C2 domain detection
23	FileDelete (archived)	Anti-forensic file deletion

8.3 Verify logging

Event Viewer → Applications and Services Logs → Microsoft → Windows → Sysmon → Operational — you should see Event ID 1 entries firing continuously as normal processes run. If empty, the service isn't running: `Get-Service Sysmon64` should show `Running`.

8.4 Point Wazuh agent at the Sysmon channel

On the Windows agent, edit `C:\Program Files (x86)\ossec-agent\ossec.conf`, add inside `<ossec_config>`:

```
<localfile>

  <location>Microsoft-Windows-Sysmon/Operational</location>

  <log_format>eventchannel</log_format>

</localfile>
```

Restart: `Restart-Service WazuhSvc`. Also add native Security log for Event IDs 4624/4625/4720/4698 used in Section 10:

```
<localfile>

  <location>Security</location>

  <log_format>eventchannel</log_format>

</localfile>
```

9. Linux Logging

9.1 Auditd (kernel-level syscall auditing)

```
sudo apt install auditd audispd-plugins -y
```

```
sudo systemctl enable --now auditd
```

Add rules relevant to this lab (`/etc/audit/rules.d/audit.rules`):

```
-w /etc/passwd -p wa -k identity_change      # user account changes
```

```
-w /etc/shadow -p wa -k identity_change
```

```
-w /etc/sudoers -p wa -k privilege_escalation
```

```
-w /var/log/auth.log -p wa -k auth_log_tamper
```

```
-a always,exit -F arch=b64 -S execve -k command_execution # log every exec syscall
```

Load: `sudo augenrules --load`

9.2 Syslog / Authentication logs

`/var/log/auth.log` (Debian/Ubuntu) already logs SSH authentication events natively — no extra config needed, just ensure the Wazuh agent reads it:

```
<localfile>
```

```
<log_format>syslog</log_format>
```

```
<location>/var/log/auth.log</location>
```

```
</localfile>
```

```
<localfile>
```

```
<log_format>syslog</log_format>
```

```
<location>/var/log/syslog</location>
```

```
</localfile>
```

9.3 SSH-specific hardening for realistic brute-force telemetry

```
sudo nano /etc/ssh/sshd_config
```

ensure: LogLevel VERBOSE (logs the key fingerprint on auth attempts, needed for Section 10's brute-force detection)

```
sudo systemctl restart sshd
```

9.4 How Linux logs reach the SIEM

auditd/syslog write to /var/log/* → Wazuh agent's <localfile> stanzas tail these files →

agent forwards over TCP 1514 to manager → manager decodes via built-in "sshd" and "auditd"

decoders → matches against ossec_rules.xml (e.g., rule 5710 = "sshd authentication failed")
→

alert indexed → visible in dashboard, same as Windows events

Both OS's logs land in the **same** Wazuh index and dashboard — this is the actual value of a SIEM: one pane of glass regardless of source OS.

10. MISP Installation (added — this was previously only referenced, never actually documented in this repo)

Audit note: earlier versions of this project's documentation set mentioned MISP throughout the architecture and detection sections but never included its actual install steps — the repo could not be built end-to-end from its own files. This section fixes that.

10.1 Install (Docker method)

```
git clone https://github.com/MISP/misp-docker.git
```

```
cd misp-docker
```

```
cp template.env .env
```

edit .env: set BASE_URL=https://192.168.56.6, set MYSQL/ADMIN passwords — do not leave defaults

docker compose up -d

First boot takes several minutes (DB migrations running). Access <https://192.168.56.6>, default login admin@admin.test / [admin](#) — **change this immediately** under My Profile. Variable names in .env change across MISP releases — cross-check <https://github.com/MISP/misp-docker> at build time rather than trusting this list as exhaustive.

10.2 Pull free threat feeds

1. **Sync Actions** → **List Feeds**
2. Enable **abuse.ch URLhaus** and **abuse.ch ThreatFox** (free, public, MISP-format).
3. **Fetch and store all feed data.**
4. Cron a re-pull every 6 hours so the feed doesn't go stale mid-project.

10.3 Generate an API key for Wazuh to query

Administration → **List Auth Keys** → **Add authentication key** — copy it immediately, it's shown once.

10.4 Install the Wazuh–MISP integration script

There is **no default Wazuh-MISP script** — it must be sourced and reviewed manually:

```
sudo curl -fsSL -o /var/ossec/integrations/custom-misp.py \
```

```
https://raw.githubusercontent.com/juaromu/wazuh-misp/main/custom-misp.py
```

```
test -s /var/ossec/integrations/custom-misp.py && echo "download OK" || echo "download FAILED — check URL/network"
```

```
sudo chmod 750 /var/ossec/integrations/custom-misp.py
```

```
sudo chown root:wazuh /var/ossec/integrations/custom-misp.py
```

-f makes curl fail loudly on a bad HTTP response instead of silently saving an error page as if it were the script; **test -s** confirms the file isn't empty before wiring it in.

Before running this script with root:wazuh permissions, open and read it. Two real issues have been found in community forks of this exact script:

1. **TLS validation disabled** — `grep -n "verify=False" /var/ossec/integrations/custom-misp.py`; if found, replace with a real cert path: `verify="/etc/wazuh/misp-ca.pem"`.
2. **Hardcoded API key expected in source** — acceptable for a quick test, but pull it from the environment instead before submitting/publishing this repo:

```
import os

misp_api_auth_key = os.environ.get("MISP_API_KEY")
```

10.5 Wire it into ossec.conf

```
<integration>
```

```
<name>custom-misp</name>
```

```
<group>sysmon_event_22,sysmon_event1,sysmon_event3,syscheck,sophos</group>
```

```
<alert_format>json</alert_format>
```

```
</integration>
```

This fires MISP enrichment on Sysmon DNS/process/network events, file-integrity changes, **and Sophos perimeter alerts** (see file 08) — group-based triggering, not hardcoded rule IDs, so it survives you adding new rules later without editing this block again. Restart: `sudo systemctl restart wazuh-manager`.

10.6 Verify before trusting it

```
sudo sed -i 's/^integrator.debug=.*integrator.debug=2/'  
/var/ossec/etc/local_internal_options.conf 2>/dev/null || echo "integrator.debug=2" | sudo tee -a  
/var/ossec/etc/local_internal_options.conf
```

```
sudo systemctl restart wazuh-manager
```

```
sudo tail -f /var/ossec/logs/ossec.log | grep -i misp
```

Trigger a test event, confirm either a successful POST/response cycle or a specific Python traceback in this log — don't assume it's working just because the manager restarted cleanly.

Continue to: [03-Detection-Engineering.md](#) for the 20 core detections with Sigma rules and MITRE mapping, or [08-Sophos-Firewall-Integration.md](#) for the perimeter layer.

